

EQBuddy

Static Code Analysis: Game Interaction and Ban-Risk Review

Professional technical assessment of source-code behaviors that interact with EverQuest, the local operating environment, and the Mac/CrossOver compatibility layer.

Repository: DranakCorps-bot/EQBuddy

Branch reviewed: main

Snapshot: 30fdeec92ad689191e34a6566d2719f3df643434

Assessment type: Static source-code review

Primary question: Whether EQBuddy performs game interaction beyond read-only consumption of game-generated files, including memory/process access, network decoding, hooks/injection, input automation, or other potentially detectable behaviors.

Principal conclusion

No evidence was found in the reviewed source of EverQuest process-memory reading/writing, DLL injection, process hooking, packet capture/decoding, or automated game input. However, EQBuddy is not strictly a log-only reader: it reads additional game-generated/map files, writes eqclient.ini, can modify log files, and on Mac/CrossOver writes Wine configuration scoped specifically to eqgame.exe. The Mac/CrossOver path therefore creates a potentially detectable EQ-specific configuration artifact, although this analysis cannot establish that Daybreak detected it or that it caused any ban.

1. Executive Summary

This review examined EQBuddy's current source tree for technical behaviors that could cross a common MMO third-party-tool boundary: direct access to the game process, memory inspection or modification, code injection, network interception, protocol decoding, automated input, or modification of game-owned files and execution environment.

The reviewed code is largely an external parser/overlay architecture. Game state is primarily derived from EverQuest text logs and user-generated output files. The review did not identify the classic mechanisms used by invasive game tools: OpenProcess/ReadProcessMemory-style access, injected code, game socket interception, packet decoders, or SendInput/PostMessage-style automation directed at EverQuest.

Three areas are nonetheless relevant to a ban-risk analysis. First, EQBuddy writes Log=1 into EverQuest's eqclient.ini. Second, it can truncate, move, archive, and recreate EverQuest log files. Third, the optional Mac/CrossOver overlay path writes a Wine AppDefaults registry value specifically under eggame.exe and relies on an optional patched Wine/CrossOver Mac driver to change fullscreen window behavior. These are materially different from passive log parsing.

Area	Observed	Assessment	Risk
Process memory	No direct game-memory APIs found	No evidence of memory inspection/modification	Low
Injection / hooks	No game-process injection or SetWindowsHookEx path found	UI hooks affect EQBuddy's own windows/messages	Low
Network interception	No packet capture or EQ protocol decoding found	Companion networking is EQBuddy's own LAN service	Low
Input automation	No SendInput/PostMessage-style game automation found	Game commands are copied for the user, not executed	Low
Game files	Logs, inventory export, achievements export, maps, eqclient.ini	Broader than 'log only'	Medium
eqclient.ini	Read and written	Direct modification of game-owned configuration	Medium
Mac/CrossOver	Wine registry value scoped to eggame.exe; optional driver patch	Potentially detectable EQ-specific environment artifact	Medium-High

1.1 What this report does not prove

- It does not prove what Daybreak's anti-cheat or enforcement systems actually inspect.
- It does not prove that any observed behavior was detected.
- It does not prove that EQBuddy caused a specific ban.
- It does not verify that a distributed installer/binary exactly matches the reviewed source snapshot.
- It does not cover historical versions or deleted functionality unless present in the reviewed snapshot.

2. Scope and Methodology

The analysis was performed as a static source review of the repository's current main-branch snapshot identified above. Searches were performed across the source tree for high-risk process, memory, injection, packet, input, filesystem, registry, native-interopability, and networking patterns. Relevant implementation files were then read to determine whether matched APIs were directed at EverQuest, EQBuddy itself, the operating system, or unrelated supporting services.

2.1 High-risk categories reviewed

- Process access and process-handle use
- Memory read/write and remote allocation
- DLL/code injection and Windows hook mechanisms
- Packet capture, raw socket interception, or EverQuest protocol decoding
- Keyboard/mouse injection and command automation
- Game-window manipulation versus EQBuddy-window manipulation
- Reads/writes/enumeration under the EverQuest installation directory
- Registry access, including configuration scoped to eqgame.exe
- Native dependencies capable of providing hidden memory/packet/input functionality
- Local and outbound networking implemented by EQBuddy

2.2 Confidence and limitations

Confidence is high for the specific source behaviors documented below because the relevant implementations are direct and readable. Confidence is lower for statements of absence than for positive findings: static source search can establish that common invasive mechanisms were not found in the reviewed tree, but only binary inspection and runtime tracing can prove what a particular released executable actually does.

3. Verified Game-Adjacent File Access

3.1 EverQuest log files

LogWatcher tails eqlog_*.txt using a normal FileStream opened with FileAccess.Read and sharing that permits the game to continue writing. It seeks from a maintained byte offset and parses newly appended lines. This is the primary game-data path.

```
new FileStream(path, FileMode.Open, FileAccess.Read, FileShare.ReadWrite | FileShare.Delete)
```

Separate maintenance functionality can also truncate, move, archive, and recreate log files. This is not a hidden game-state mechanism, but it means the application is not strictly read-only toward the EverQuest installation.

3.2 eqclient.ini

EqConfig locates eqclient.ini in the EverQuest installation root and can modify the [Defaults] section so that Log=1. The function avoids the write while eqgame.exe is detected as running, but the file is still a game-owned configuration file and the operation is an explicit write.

```
File.ReadAllLines(ini) -> set "Log=1" -> File.WriteAllLines(ini, lines)
```

Risk significance

This is the clearest Windows/default-install behavior that weakens a 'read-only game files' claim. Removing it would materially simplify the product's defensibility.

3.3 Inventory export

InventoryFile searches the EverQuest installation root for the newest <Character>_<server>-Inventory.txt file and reads it. The file is generated by EverQuest in response to the player's /outputfile inventory command. EQBuddy parses the export; it does not issue the command itself.

3.4 Achievements export

The user interface offers /outputfile achievements as copyable text. The user then selects the generated achievements dump with a normal file picker, which defaults to the EverQuest installation root. EQBuddy reads the selected text file and imports relevant completion data.

3.5 Zone map files

ZoneMap derives the game's maps directory as a sibling of the Logs folder, enumerates .txt map files, resolves the expected zone short name, and reads the selected map file with File.ReadLines(). The parser accepts classic EverQuest map L/P line formats.

```
<EverQuest root>\maps\*.txt -> Directory.EnumerateFiles / File.ReadLines
```

3.6 Files not found in the reviewed runtime path

The review did not identify runtime reads of eqgame.exe itself, game DLLs, eqhost.txt, character INI files, UI XML, S3D/EQG archives, texture/model assets, or other binary game data. This is an absence finding for the reviewed snapshot, not a guarantee for historical releases.

4. Process, Memory, Injection, Network, and Input Analysis

4.1 EverQuest process interaction

EQBuddy calls `Process.GetProcessesByName("eqgame")` to determine whether the game is running. That check is used to suppress configuration/log-maintenance operations while the game is active. The reviewed source did not show a subsequent process handle, module inspection, or memory access path.

```
Process.GetProcessesByName("eqgame").Length > 0
```

4.2 Memory access and injection

Repository-wide searches did not identify the standard process-memory/injection primitives expected in a memory-reading or injected game tool, including `OpenProcess`, `ReadProcessMemory`, `WriteProcessMemory`, `VirtualAllocEx`, `CreateRemoteThread`, or `SetWindowsHookEx` directed at EverQuest.

Finding

No evidence was found that EQBuddy opens EverQuest for process-memory access or injects executable code into the game.

4.3 Native Windows APIs that are present

The Windows UI does use user32 APIs, but the reviewed implementations are directed at EQBuddy's own windows or global hotkey registration. `RegisterHotKey/UnregisterHotKey` receive user-configured shortcuts. `GetWindowLong/SetWindowLong` apply styles such as `WS_EX_NOACTIVATE` and `WS_EX_TOOLWINDOW` to EQBuddy overlay windows.

A WPF 'WndProc hook' exists for processing `WM_HOTKEY` messages on EQBuddy's own window. This terminology can look suspicious in a superficial grep, but it is not a game-process hook.

4.4 Input automation

The source centralizes useful EverQuest command strings such as `/outputfile inventory`, `/outputfile achievements`, `/loc`, and `/doability 1`. The implemented workflow copies these commands to the clipboard for the player. The review did not find `SendInput`, `keybd_event`, `mouse_event`, `PostMessage`, or similar game-input injection paths.

Finding

EQBuddy assists the user with command text but does not appear to execute those commands inside EverQuest.

4.5 Networking

EQBuddy contains ordinary application networking for wiki lookups, update checks/downloads, and an optional LAN HTTP/WebSocket companion server. The companion server exposes EQBuddy-derived data to a paired phone; it is not positioned between EverQuest and Daybreak and no packet-capture or EverQuest protocol-decoding library/path was identified.

5. Mac/CrossOver Finding: eqgame.exe-Specific Wine Configuration

The Mac/CrossOver overlay path is the most notable environment-level behavior identified in the review. When the Windows build is running under Wine, WineOverlay.Configure writes Wine AppDefaults registry values. One of those values is specifically scoped to eqgame.exe.

```
HKCU\Software\Wine\AppDefaults\eqgame.exe\Mac Driver
KeepFullscreenWhenInactive = "Y" or "N"
```

5.1 Why this is technically significant

The implementation calls the helper for eqgame.exe whenever Configure runs under Wine. The helper creates/opens the registry subkey and writes Y or N based on the setting. Therefore, a false setting still creates or updates an eqgame.exe-specific registry artifact rather than simply leaving the key untouched.

```
SetMacDriverKey("eqgame.exe", "KeepFullscreenWhenInactive", settings.WineKeepGameFullscreen)
key?.SetValue(name, on ? "Y" : "N")
```

5.2 Optional CrossOver/Wine driver patch

The repository also documents an optional Mac/CrossOver setup that patches the winemac driver. The patch changes how fullscreen window levels are handled so EQBuddy can remain above the fullscreen game and, optionally, so eqgame.exe can remain visually fullscreen when inactive.

This does not modify the eqgame.exe binary and does not constitute the memory-injection behavior typically associated with a game cheat. Nevertheless, it changes the compatibility layer controlling the game's window and associates configuration directly with eqgame.exe.

5.3 Detectability

Because EverQuest is running inside the same Wine/CrossOver environment, software executing in that environment could, in principle, inspect registry configuration or other aspects of the Wine runtime. Accordingly, the eqgame.exe-specific AppDefaults entry is a potentially detectable artifact.

Important distinction

Potentially detectable does not mean known to be detected. The EQBuddy source cannot reveal Daybreak's anti-cheat telemetry or enforcement logic, and this review provides no evidence that the registry key was inspected or caused a ban.

5.4 Relative concern

For a user who ran EQBuddy on Mac/CrossOver, this path deserves substantially more scrutiny than ordinary log parsing because it creates an EQ-specific configuration artifact outside EQBuddy's own application data. For a normal Windows user, this Wine-gated code path returns early and is not active.

6. Risk Assessment

Finding	Evidence	Interaction type	Ban-risk relevance	Priority
Read EQ logs	FileAccess.Read tailing	Passive game-generated text	Generally lowest technical concern	Low
Read inventory export	File.ReadLines on /outputfile inventory dump	Passive user-requested export	Low technical invasiveness	Low
Read achievements export	User-selected text dump	Passive user-requested export	Low technical invasiveness	Low
Read maps	Enumerate/read maps/*.txt	Read game/map data	Broader than log-only	Low-Med
Write eqclient.ini	File.WriteAllLines after forcing Log=1	Modify game configuration	Direct game-file modification	Medium
Modify EQ logs	truncate/move/recreate/archive	Modify game-generated files	Not cheating, but not read-only	Medium
Check eqgame process	GetProcessesByName	Process enumeration only	Observable but non-invasive	Low
Wine eqgame AppDefaults	CreateSubKey + SetValue	EQ-specific runtime configuration	Potentially detectable artifact	High
Patched winemac driver	Optional documented patch	Compatibility-layer modification	Strong policy/optics concern on Mac	High

6.1 Causation assessment

Nothing in the reviewed source supports a conclusion that EQBuddy definitely caused a ban. A ban decision could be based on the mere presence/use of an unapproved third-party tool, a specific feature, a heuristic classification, another program on the machine, historical EQBuddy behavior, or unrelated account activity. Static source review is useful for identifying plausible technical triggers, not proving enforcement causation.

7. Recommendations

If the objective is to minimize technical interaction with EverQuest and make the tool's behavior easy to explain and audit, the recommended target architecture is strict one-way ingestion of user-visible, game-generated text.

Remove eqclient.ini modification. Do not force Log=1. Require the player to enable logging in game. This removes the most obvious default-platform write to a game-owned configuration file.

Move all log maintenance outside the product or make it strictly opt-in. Prefer never modifying files in the EverQuest directory. If archival is retained, copy out rather than truncate/move/recreate the source.

Eliminate eqgame.exe-specific Wine registry writes. Do not create AppDefaults entries for eqgame.exe. If the setting is false, especially avoid writing a persistent 'N' artifact.

Retire or isolate the CrossOver driver patch. For a conservative build, remove compatibility-layer modifications tied to EverQuest fullscreen behavior.

Offer a 'strict read-only' build/profile. Limit EverQuest access to FileAccess.Read on eqlog_*.txt and, if necessary, explicit user-selected exports. No install-root writes or process-specific configuration.

Verify release binaries. Hash and inspect the exact executable/installer used by any banned account; compare assemblies/imports to the matching source revision.

Audit Git history/releases. Review prior tagged releases and deleted code for any historical packet, memory, input, or process-interaction functionality.

Add a machine-verifiable I/O contract. Automated tests should fail if production code adds writes under the EQ root, process-handle access, input-injection APIs, packet-capture dependencies, or eqgame-specific registry writes.

Conservative target state

EverQuest\Logs\eqlog_*.txt --FileAccess.Read--> EQBuddy. All EQBuddy settings, caches, databases, archives, and telemetry-free application state remain outside the EverQuest installation.

Appendix A - Evidence Map

The following source files contain the principal behaviors relied upon in this assessment. Line numbers are not embedded because the repository is actively changing; file/function names and the reviewed commit identify the evidence more reliably.

Source	Relevant behavior
src/EQBuddy.Core/LogWatcher.cs	Log tailing, install/log-folder discovery, read-only uninstall-registry lookup, process-independent log ingestion.
src/EQBuddy.Core/EqConfig.cs	eggame/GINA/GamParse process-name checks; eqclient.ini read/write; log truncation, archive, split, move, and recreation.
src/EQBuddy.Core/InventoryFile.cs	Discovery and read of <Character>_<server>-Inventory.txt generated by /outputfile inventory.
src/EQBuddy.Core/ZoneMap.cs	Derives <EQ root>/maps, enumerates .txt files, and reads map content.
src/EQBuddy/QuestChecklistView.cs	Achievements dump file picker/read and copy-to-clipboard command workflow.
src/EQBuddy.UI.Shared/GameCommands.cs	Centralized in-game command strings presented to the user.
src/EQBuddy/HotkeyManager.cs	RegisterHotKey/UnregisterHotKey and WM_HOTKEY handling for EQBuddy actions.
src/EQBuddy/NoActivate.cs	Get/SetWindowLong calls applied to EQBuddy's own overlay HWND.
src/EQBuddy/WineOverlay.cs	Wine detection, EQBuddy window styles, Wine AppDefaults writes for EQBuddy.exe and eggame.exe.
docs/CrossOver-macOS-overlay.md	Documented optional winemac driver patch and fullscreen behavior.
src/EQBuddy.Companion/CompanionServer.cs	LAN TCP/WebSocket service for EQBuddy Mobile; unrelated to EverQuest network interception.
src/EQBuddy.Core/EQBuddy.Core.csproj	Core dependency set; no memory/packet/input-injection package identified.

Appendix B - Search Categories with No Positive Finding

The following mechanisms were specifically sought because they would materially change the assessment. No positive runtime finding directed at EverQuest was identified in the reviewed snapshot:

- OpenProcess / process-handle acquisition for eggame.exe
- ReadProcessMemory / WriteProcessMemory
- VirtualAllocEx / CreateRemoteThread
- DLL injection into EverQuest
- SetWindowsHookEx-based game hooking
- SharpPcap / PacketDotNet / WinDivert-style packet capture
- EverQuest protocol or packet decoding
- SendInput / keybd_event / mouse_event
- PostMessage / SendMessage automation directed at the game

- Game DLL/executable binary parsing as a runtime data source

Appendix C - Reference URLs

Repository: <https://github.com/DranakCorps-bot/EQBuddy>

EqConfig.cs: <https://github.com/DranakCorps-bot/EQBuddy/blob/main/src/EQBuddy.Core/EqConfig.cs>

LogWatcher.cs: <https://github.com/DranakCorps-bot/EQBuddy/blob/main/src/EQBuddy.Core/LogWatcher.cs>

InventoryFile.cs: <https://github.com/DranakCorps-bot/EQBuddy/blob/main/src/EQBuddy.Core/InventoryFile.cs>

ZoneMap.cs: <https://github.com/DranakCorps-bot/EQBuddy/blob/main/src/EQBuddy.Core/ZoneMap.cs>

WineOverlay.cs: <https://github.com/DranakCorps-bot/EQBuddy/blob/main/src/EQBuddy/WineOverlay.cs>

Mac/CrossOver overlay documentation:

<https://github.com/DranakCorps-bot/EQBuddy/blob/main/docs/CrossOver-macOS-overlay.md>

SECURITY.md: <https://github.com/DranakCorps-bot/EQBuddy/blob/main/SECURITY.md>

End of report